

NeuroScan AI

Brain Abnormality Detection Platform

Complete Development Plan — Phases, Verticals & Task Assignment

TARUN	JEMIN
Frontend & Backend Electron · Node.js · React	AI / ML Engineering Mamba · PyTorch · LLM

7	5	~18	2
Phases	Layers	Months	Engineers

TARUN — Frontend / Backend	JEMIN — AI / ML	BOTH — Shared	■ — Parallel Phase	★ CRITICAL — Gate Item
----------------------------	-----------------	---------------	--------------------	------------------------

System Architecture Overview

1	INPUT PIPELINE	Raw K-Space · DICOM · Metadata · Multi-Sequence MRI
2	RECONSTRUCTION	IFFT · Coil Combination · Motion Correction · Denoising · Artifact Detection
3	FOUNDATION MODEL	K-Space Encoder (Complex Mamba) + Image Encoder (ViT/CNN) → Fusion Backbone
4	SEGMENTATION	Tumor Masks · Lesion Masks · Tissue Segmentation · Volume Calculation
5	DETECTION MODELS	Tumor · Stroke · Hemorrhage · Microbleeds · MS Lesions
6	DISEASE INTELLIGENCE	Classification · Severity · Risk Scores · Disease Subtypes
7	RADIOLOGY AGENT	LLM Reasoning · Differential Diagnosis · Report Drafting · Recommendations
8	REPORTING LAYER	Findings · Measurements · DICOM-SR · Evidence Links
9	VISUALIZATION LAYER	MRI Viewer · Axial/Coronal/Sagittal · Heatmaps · 3D Brain · Charts/Graphs

■ Parallel-capable phases are marked. Jemin can begin AI/ML work from Phase 1 onwards in parallel with infrastructure setup by Tarun.

Recommended Tech Stack

FRONTEND / ELECTRON

Framework: Electron 29 + React 18	UI Library: Tailwind CSS + shadcn/ui	State: Zustand / Redux Toolkit
3D Rendering: Three.js / VTK.js	MRI Viewer: Cornerstone3D (DICOM)	Charts: Recharts / D3.js

BACKEND (Node.js)

Runtime: Node.js 20 LTS + TypeScript	API: Express.js / Fastify	Database: PostgreSQL + Prisma ORM
File Store: MinIO (DICOM / K-Space)	Queue: BullMQ + Redis	Auth: JWT + bcrypt

AI / ML (Python)

Framework: PyTorch 2 + CUDA	K-Space Model: Mamba (state-space) + complex FFT	Image Model: ViT / SwinTransformer
Segmentation: nnU-Net / MedSAM	LLM Agent: GPT-4o / Claude API + LangChain	Serving: FastAPI + TorchServe

INFRASTRUCTURE

Containerize: Docker + Docker Compose	IPC Bridge: Electron IPC / REST to Python	DICOM Std: Orthanc DICOM Server
Monitoring: Prometheus + Grafana	Testing: Jest (FE) + Pytest (AI)	Packaging: electron-builder

Project Setup & Research Foundation

Duration: 2–3 weeks

SEQUENTIAL PHASE

Establish the development environment, repository structure, CI pipeline, and shared conventions. Jemin conducts deep-dive literature review on K-Space abnormality detection and Mamba architecture. Tarun bootstraps the Electron shell, Node.js API skeleton and database schema.

Verticals

- **Repo & CI/CD:** Monorepo (pnpm workspaces): electron-app, backend, ai-service. GitHub Actions for lint/test.
- **Dev Environment:** Docker Compose: PostgreSQL, Redis, MinIO, Python FastAPI. All services local.
- **Research Baseline:** Survey: K-Space ML papers, Mamba architecture, nnU-Net, MedSAM, DICOM standards.
- **Data Strategy:** Identify public datasets: fastMRI, BraTS, ATLAS. DICOM anonymisation pipeline plan.
- **Architecture Doc:** Finalise inter-service contracts (REST/IPC), data schemas, API contracts.

Task Breakdown

Who	Task	Notes / Tech
TARUN	Initialise monorepo with pnpm workspaces + ESLint/Prettier	GitHub, pnpm
TARUN	Electron shell: BrowserWindow, IPC bridge skeleton, auto-updater scaffold	Electron 29
TARUN	Node.js API: Express + TypeScript + Prisma schema (Patient, Study, Report)	PostgreSQL 16
TARUN	Docker Compose: all backing services + hot-reload	Docker
TARUN	MinIO bucket structure for DICOM/K-Space files	MinIO
JEMIN	Literature review: K-Space anomaly detection, sub-voxel microbleeds	Papers/arXiv
JEMIN	Evaluate Mamba variants: S4, Mamba-2, VMamba for 3D MRI	PyTorch
JEMIN	Set up Python AI-service: FastAPI + TorchServe scaffold	FastAPI
JEMIN	Identify training datasets: fastMRI, BraTS 2024, ATLAS 2.0	Data survey
BOTH	Define REST API contracts between backend ↔ AI-service	OpenAPI spec
BOTH	Define IPC message schema: Electron ↔ Backend	TypeScript types

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	Monorepo initialised, all devs can run full stack locally
★ CRITICAL	Docker Compose: Postgres, Redis, MinIO, FastAPI all healthy
✓	Electron window launches, IPC bridge responds to ping
★ CRITICAL	API OpenAPI spec reviewed and approved by both
✓	Literature review doc written and shared
★ CRITICAL	Training dataset access confirmed (at least fastMRI + BraTS)
✓	GitHub Actions CI: lint + test passes on main

Input Pipeline & Reconstruction

Duration: 3–4 weeks

SEQUENTIAL PHASE

Build the complete data ingestion path: raw K-Space upload, DICOM parsing, multi-sequence MRI handling, and the reconstruction pipeline (IFFT, coil combination, motion correction, denoising, artifact detection). This is the data foundation all models depend on.

Verticals

- **DICOM Ingestion:** Parse DICOM files, extract metadata, store pixel data and K-Space raw in MinIO.
- **K-Space Loader:** Read raw K-Space (.dat / .h5), apply IFFT, coil combination (RSS/SENSE).
- **Motion Correction:** Retrospective rigid motion correction on reconstructed images.
- **Denoising:** Deep learning denoiser (BM3D baseline + DnCNN fine-tuned on fastMRI).
- **Artifact Detection:** Classify acquisition artifacts: ghosting, wrap-around, zipper noise.
- **API Endpoints:** POST /studies/upload, GET /studies/{id}/sequences, GET /kpace/{id}/reconstructed.

Task Breakdown

Who	Task	Notes / Tech
TARUN	DICOM upload endpoint: multipart, DICOM parse (dcmjs), store metadata in DB	dcmjs, Prisma
TARUN	MinIO upload integration: DICOM pixel data + raw K-Space files	MinIO SDK
TARUN	Background job queue: BullMQ — process study on upload	BullMQ + Redis
TARUN	REST endpoints: studies CRUD, sequences list, reconstruction status	Express
JEMIN	K-Space reader: support .dat (Siemens twix), .h5 (fastMRI), custom numpy	PyTorch, h5py
JEMIN	Reconstruction: IFFT + RSS coil combination + phase correction	NumPy/PyTorch
JEMIN	Motion correction module: phase-correlation rigid registration	SimpleITK
JEMIN	Denoiser: BM3D baseline, fine-tune DnCNN on fastMRI validation set	PyTorch
JEMIN	Artifact detection classifier (CNN): ghosting, wrap, spike noise	torchvision
BOTH	End-to-end test: upload DICOM → reconstruct → artifact report returned	Pytest + Jest

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	DICOM upload, parse, and DB store working end-to-end
★ CRITICAL	K-Space IFFT reconstruction produces clean magnitude images
★ CRITICAL	Coil combination (RSS) correct on multi-coil fastMRI data
★ CRITICAL	Denoiser PSNR > 34 dB on fastMRI validation set
✓	Artifact detector accuracy > 85% on held-out set
✓	BullMQ job processes study < 30 s for typical 3T scan
★ CRITICAL	All API endpoints returning correct responses, covered by integration tests

Foundation AI Model — K-Space + Image Encoders ■ PARALLEL

Duration: 6–8 weeks

■ PARALLEL PHASE

Core AI phase. Jemin builds the dual-encoder Foundation Model: a Complex Mamba K-Space Encoder and a ViT/CNN Image Encoder, fused via a Fusion Backbone. Tarun builds the AI-service bridge, model serving infrastructure, and the "gating" logic (if K-Space detects nothing → skip image model). This phase CAN start in parallel with Phase 1 for the model research/training prep tasks.

Verticals

- **K-Space Encoder:** Complex-valued Mamba SSM operating on raw K-Space frequency data. Detects sub-voxel anomalies invisible in image domain.
- **Image Encoder:** SwinTransformer-based encoder on reconstructed magnitude images. Handles tumors, large lesions well-visualised in image space.
- **Fusion Backbone:** Cross-attention fusion: K-Space features + Image features → shared latent. Gate: if K-Space anomaly score < threshold, bypass image encoder.
- **Training Pipeline:** Multi-task learning: reconstruction quality + anomaly presence + disease classification. Curriculum: clean → noisy → pathological.
- **Model Serving:** TorchServe handler exposed via FastAPI. Electron calls Node.js → Node calls FastAPI → TorchServe.
- **Gating Logic:** K-Space model produces anomaly_detected bool + confidence. Backend routes to image model only if anomaly_detected=True.

Task Breakdown

Who	Task	Notes / Tech
JEMIN	Implement Complex Mamba encoder: complex-valued SSM on K-Space slices	<i>PyTorch, mamba-ssm</i>
JEMIN	Pre-train K-Space encoder on fastMRI reconstruction task (self-supervised)	<i>fastMRI</i>
JEMIN	Fine-tune K-Space encoder: anomaly detection head (microbleeds, lesions)	<i>BraTS, ATLAS</i>
JEMIN	Implement SwinTransformer image encoder + classification head	<i>timm library</i>
JEMIN	Cross-attention fusion module: K-Space + Image feature alignment	<i>PyTorch nn.MultiheadAttention</i>
JEMIN	Gating mechanism: anomaly_score threshold, configurable per pathology	<i>PyTorch</i>
JEMIN	Training loop: multi-task loss (recon + anomaly + classify), mixed precision	<i>PyTorch Lightning</i>
JEMIN	Validation: AUC > 0.90 on microbleed detection, sensitivity > 85%	<i>scikit-learn</i>
JEMIN	TorchServe model archive (.mar) + custom handler	<i>TorchServe</i>
TARUN	FastAPI AI-service: /predict endpoint, request/response schema	<i>FastAPI, Pydantic</i>
TARUN	Node.js AI client: calls FastAPI, handles gating response, stores results	<i>Axios</i>
TARUN	DB schema: ModelResult, AnomalyDetection, GatingDecision tables	<i>Prisma</i>
TARUN	BullMQ worker: trigger AI inference on study upload completion	<i>BullMQ</i>
BOTH	Integration test: DICOM → reconstruct → K-Space model → gating → response	<i>Pytest + Jest</i>

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	K-Space encoder trains without NaN loss on complex-valued data
★ CRITICAL	K-Space anomaly AUC > 0.88 on ATLAS microbleed validation set
★ CRITICAL	Gating: correctly skips image model when K-Space score < threshold
★ CRITICAL	Image encoder + fusion loss converges, acc > 80% on BraTS holdout
✓	TorchServe serving latency < 3 s per study on GPU
★ CRITICAL	Node.js AI client sends request and stores result in DB correctly
★ CRITICAL	Full pipeline test: upload → reconstruct → K-Space → (gate) → image → DB

Detection Models & Segmentation

Duration: 5–6 weeks

SEQUENTIAL PHASE

Build specialised detection models (Tumor, Stroke, Hemorrhage, Microbleeds, MS Lesions) and segmentation models that produce voxel-level masks for each detected abnormality. Volume calculations and spatial metadata feed the 3D visualisation and disease intelligence layers.

Verticals

- **Detection Ensemble:** Per-pathology classifiers built on top of Foundation Model features. Each model outputs: present/absent + bounding region + confidence.
- **Tumor Detection:** Multi-class: glioma / meningioma / metastasis. Uses BraTS 2024 labels.
- **Microbleed Detection:** Most challenging. K-Space gating is essential here. CMB-3D dataset + augmentation.
- **MS Lesion Detection:** White matter lesion detector. ISBI challenge data. T2/FLAIR emphasis.
- **Segmentation (nnUNet):** nnU-Net 3D full-res for tumor and large lesion masks. MedSAM for interactive refinement.
- **Volume & Spatial:** Lesion volume (ml), centroid (voxel + MNI coords), cluster count, spatial distribution map.

Task Breakdown

Who	Task	Notes / Tech
JEMIN	Tumor detector: fine-tune from Foundation backbone on BraTS 2024	BraTS 2024
JEMIN	Stroke/hemorrhage detector: ICH dataset fine-tuning	RSNA ICH
JEMIN	Microbleed detector: CMB-3D + synthetic augmentation (sphere phantoms)	CMB-3D, PyTorch
JEMIN	MS lesion detector: ISBI 2015 challenge fine-tuning	ISBI challenge
JEMIN	nnU-Net training: tumor + large hemorrhage segmentation masks	nnU-Net v2
JEMIN	MedSAM integration: interactive prompt-based segmentation refinement	MedSAM
JEMIN	Volume calculation: voxel count × voxel spacing → ml, MNI registration	SimpleITK, ANTs
JEMIN	Cluster analysis: connected components, centroid extraction, spatial pattern	scipy.ndimage
TARUN	DB schema: Detection, Segmentation, LesionMask, VolumeMetric tables	Prisma
TARUN	FastAPI endpoints: /detect/{study_id}, /segment/{detection_id}	FastAPI
TARUN	Node.js orchestrator: call detect → if finding → call segment → store	Node.js
TARUN	Mask storage: compressed NIfTI .nii.gz in MinIO, metadata in DB	nifti-reader-js
BOTH	Accuracy benchmarks documented: per-pathology AUC, Dice score table	MLflow

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	Tumor detector: AUC > 0.93 on BraTS 2024 holdout
★ CRITICAL	Microbleed detector: sensitivity > 80%, specificity > 85%
★ CRITICAL	MS lesion detector: Dice > 0.65 on ISBI holdout
★ CRITICAL	nnU-Net tumor segmentation: Dice > 0.85 on BraTS holdout
✓	Volume calculation within 5% of ground truth on phantom data

★ CRITICAL	Masks stored as NIfTI in MinIO, metadata in DB, retrievable via API
★ CRITICAL	Orchestration: detect → segment pipeline runs without manual intervention

Disease Intelligence & Radiology Agent

Duration: 4–5 weeks

SEQUENTIAL PHASE

Add clinical intelligence on top of raw detections. The Disease Intelligence layer classifies disease subtypes, assigns severity scores and risk scores. The Radiology Agent (LLM-powered) reasons over all findings, drafts a differential diagnosis, generates a structured report, and produces recommendations. This layer is what makes the product clinically meaningful beyond a simple detection tool.

Verticals

- **Disease Classification:** Subtype classification per pathology. E.g., glioma grade I–IV, ICH subtypes (lobar/deep/IVH).
- **Severity Scoring:** Numerical severity score (0–10) per finding, derived from volume + location + count.
- **Risk Scoring:** Composite risk score: current severity + growth pattern + patient metadata (age, BP, history).
- **Radiology Agent LLM:** System-prompted GPT-4o/Claude API. Input: structured JSON of all findings. Output: differential diagnosis + report draft.
- **Report Drafting:** DICOM-SR compatible structured report. Sections: Clinical Info, Technique, Findings, Impression, Recommendations.
- **Evidence Links:** Each report statement linked to the detection/segmentation that supports it (traceability).

Task Breakdown

Who	Task	Notes / Tech
JEMIN	Disease subtype classifier: fine-tune on subtype-labelled datasets per pathology	<i>PyTorch</i>
JEMIN	Severity scoring function: weighted formula (volume, location, mass effect)	<i>Python</i>
JEMIN	Risk model: logistic regression / small MLP on (severity + patient metadata)	<i>scikit-learn</i>
JEMIN	LLM prompt engineering: radiology agent system prompt, finding JSON schema	<i>OpenAI / Anthropic API</i>
JEMIN	LangChain agent: multi-step reasoning, evidence retrieval, differential generation	<i>LangChain</i>
TARUN	DB schema: DiseaseClassification, SeverityScore, RiskScore, RadiologyReport	<i>Prisma</i>
TARUN	FastAPI endpoints: /classify, /severity, /risk, /report/generate	<i>FastAPI</i>
TARUN	LLM API integration in Node.js: call AI-service, stream response to frontend	<i>Node.js streams</i>
TARUN	Report export: PDF generation (structured report), DICOM-SR JSON export	<i>PDFKit</i>
TARUN	Evidence link data model: report_statement ↔ detection_id foreign key	<i>Prisma</i>
BOTH	Evaluate report quality: radiologist review of 20 sample reports	<i>Manual eval</i>

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	Disease subtype classifier accuracy > 80% per pathology
★ CRITICAL	Severity + risk scores correlate with radiologist ground truth ($r > 0.80$)
★ CRITICAL	Radiology agent produces structured differential diagnosis for all pathologies

★ CRITICAL	LLM report draft passes radiologist review (no critical errors)
✓	DICOM-SR export validates against DICOM standard tools
★ CRITICAL	Evidence links correctly trace each finding back to detection ID
✓	Report PDF generated and downloadable from Node.js endpoint

Visualization Layer — 3D Brain, MRI Viewer, Charts ■ PARALLEL

Duration: 5–6 weeks

■ PARALLEL PHASE

The full Electron frontend visualization. Three sub-verticals: (1) Cornerstone3D-based MRI viewer with multi-planar reconstruction and heatmap overlay; (2) Three.js 3D brain with lesion markers, cluster patterns, volume rendering; (3) D3.js/Recharts dashboards for disease progression, risk timeline, and growth prediction charts. This phase can be partially paralleled with Phase 4.

Verticals

- **MRI Viewer:** Cornerstone3D: axial/coronal/sagittal, WW/WL adjustment, zoom/pan, DICOM metadata panel.
- **Heatmap Overlay:** AI attention heatmap (Grad-CAM) overlaid on MRI slices. Toggle on/off per model.
- **3D Brain Model:** Three.js volumetric brain render. Lesion spheres at MNI coordinates. Color by pathology type. Interactive rotation.
- **Lesion Markers:** Click lesion → popup: type, volume, severity, evidence link, slice crosshair jump.
- **Progression Charts:** Recharts: lesion volume over time, risk score trend, predicted growth curve (3/6/12 month forecast).
- **Comparative View:** Side-by-side prior vs current study comparison in viewer.

Task Breakdown

Who	Task	Notes / Tech
TARUN	Cornerstone3D integration in Electron: DICOM load from MinIO, multi-planar view	Cornerstone3D
TARUN	Heatmap overlay: receive Grad-CAM tensor from backend, render as semi-transparent layer	Canvas API
TARUN	Three.js 3D brain: load MNI brain atlas mesh, position lesion spheres from DB coords	Three.js
TARUN	Lesion popup component: fetch detection detail, link to report section	React
TARUN	Recharts progression dashboard: volume trend, risk timeline, growth forecast	Recharts / D3
TARUN	Prior study comparison: sync scroll, WW/WL, overlay diff heatmap	Cornerstone3D
TARUN	Electron IPC: stream MRI frames and 3D data without blocking renderer	Electron IPC
JEMIN	Grad-CAM implementation: per-layer attention maps for K-Space and image encoder	PyTorch CAM
JEMIN	Growth prediction model: time-series forecast on lesion volume (LSTM / GP)	PyTorch / GPyTorch
JEMIN	Future disease risk chart data: predict risk score trajectory with CI bands	Python
BOTH	Performance test: 3D brain renders < 2 s, MRI viewer < 500 ms slice load	Electron DevTools

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	MRI viewer loads DICOM, shows axial/coronal/sagittal, WW/WL works
★ CRITICAL	Heatmap overlay correctly aligned to MRI slice for all orientations

★ CRITICAL	3D brain renders in Electron, lesion spheres at correct MNI positions
★ CRITICAL	Lesion click popup shows correct type, volume, severity from DB
★ CRITICAL	Growth prediction chart renders with 3/6/12 month forecast + confidence bands
✓	Viewer performance: no frame drops on 512x512 DICOM series
✓	Prior comparison: two studies sync-scrolled without lag

Integration, Testing & Clinical Validation

Duration: 4–5 weeks

SEQUENTIAL PHASE

End-to-end integration testing of the complete pipeline. Clinical validation with radiologist review. Performance benchmarking (latency, memory, GPU). Security audit (PHI handling, HIPAA checklist). Electron packaging and installer. Beta release to a small group of test radiologists.

Verticals

- **E2E Integration:** Full pipeline: DICOM upload → reconstruct → K-Space model → detect → segment → intelligence → report → visualize.
- **Clinical Validation:** 20+ cases reviewed by radiologist. Compare AI findings to ground truth radiology reports.
- **Performance:** GPU inference profiling. Memory usage. Electron app startup time < 5 s. Report generation < 10 s.
- **Security & Privacy:** PHI anonymisation, JWT auth, API rate limiting, audit logs, HTTPS in local network mode.
- **Packaging:** electron-builder: Windows installer (.exe), macOS DMG. Auto-update channel.
- **Documentation:** User manual for radiologists, API docs (Swagger), AI model card, data sheet.

Task Breakdown

Who	Task	Notes / Tech
TARUN	End-to-end integration test suite: 10 representative DICOM studies	<i>Jest + Supertest</i>
TARUN	Security: JWT auth, rate limiting, PHI fields encrypted at rest	<i>bcrypt, AES-256</i>
TARUN	Electron packaging: Windows + macOS builds, code signing	<i>electron-builder</i>
TARUN	Auto-updater: Squirrel.Windows / electron-updater with update server	<i>electron-updater</i>
TARUN	API documentation: Swagger/OpenAPI auto-generated from Express routes	<i>swagger-jsdoc</i>
TARUN	User manual: Electron app guide for radiologist end-users	<i>Markdown → PDF</i>
JEMIN	Clinical validation: run 20+ cases, compute sensitivity/specificity vs GT	<i>Python eval scripts</i>
JEMIN	GPU profiling: latency per model, memory peak, batch size optimisation	<i>PyTorch Profiler</i>
JEMIN	Model card: training data, intended use, limitations, bias analysis	<i>Documentation</i>
JEMIN	Grad-CAM quality review: ensure heatmaps are radiologically meaningful	<i>Manual review</i>
BOTH	Bug bash: full team stress test with edge-case DICOMs (motion, metal artefacts)	<i>Manual testing</i>
BOTH	Radiologist feedback session: collect structured feedback, prioritise fixes	<i>Feedback form</i>

Phase Gate Checklist (must complete before next phase)

★ CRITICAL	All 10 E2E integration tests pass on clean CI run
★ CRITICAL	Clinical validation: sensitivity > 85% for microbleeds on 20-case set
★ CRITICAL	Clinical validation: radiologist satisfaction score > 4/5
★ CRITICAL	PHI anonymisation verified: no patient name/ID in exported files
★ CRITICAL	Electron Windows installer builds, installs, and auto-updates cleanly

✓	macOS DMG builds and notarises without Gatekeeper warning
★ CRITICAL	Total pipeline time (upload → report) < 60 s on target GPU hardware
✓	Model card + user manual reviewed and approved

Project Timeline Overview

Phase	Name	Duration	Weeks	Parallel?	Owner
0	Setup & Research	2–3 w	W1–W3	No	Both
1	Input Pipeline & Reconstruction	3–4 w	W3–W7	No	Both
2	Foundation AI Model	6–8 w	W5–W13	YES	Both
3	Detection & Segmentation	5–6 w	W13–W19	No	Jemin
4	Disease Intelligence & LLM Agent	4–5 w	W19–W24	No	Both
5	Visualization Layer	5–6 w	W22–W28	YES	Tarun
6	Integration, Testing & Validation	4–5 w	W28–W33	No	Both

Total estimated timeline: ~33 weeks (~8 months). With parallel execution on Phase 2 and 5, effective wall-clock time is ~7 months.

Parallel Work Opportunity Map

Week Range	TARUN (Frontend/Backend)	JEMIN (AI/ML)
W1–W3	Monorepo, Electron shell, Node API, DB schema, Docker	Literature review, dataset access, AI-service scaffold
W3–W7	DICOM ingestion, MinIO, BullMQ, REST endpoints	K-Space reader, IFFT reconstruction, denoiser, artifact detector
W5–W13	AI bridge, TorchServe client, gating logic in Node	Complex Mamba encoder, ViT image encoder, fusion backbone
W13–W19	DB schema for detections, Node orchestrator, mask store	Detection models (tumor/bleed/MS), nnU-Net seg, volume calc
W19–W28	LLM streaming, report PDF, 3D brain, MRI viewer, charts	Subtype classifier, risk model, Grad-CAM, growth predictor
W28–W33	Packaging, security, E2E tests, user manual	Clinical validation, GPU profiling, model card

Datasets & Pre-trained Model References

Dataset	Use Case	Size	Access
fastMRI (NYU)	K-Space pre-training, reconstruction	~1500 subjects	Free (fastmri.org)
BraTS 2024	Tumor detection + segmentation	~2000 cases	Free (Synapse)
ATLAS 2.0	Stroke lesion segmentation	~1271 T1w	Free (USC)
RSNA ICH 2019	Intracranial hemorrhage detection	~25k CT/MRI	Free (Kaggle)
CMB-3D	Cerebral microbleed detection	~75 subjects	Request access
ISBI 2015 MS Lesions	MS lesion detection	~21 subjects	Free (ISBI)
MNI152 Brain Atlas	3D brain mesh for visualization	Template	Free (MNI)
IXI Dataset	Normal brain T1/T2/PD for baseline	~600 subjects	Free (brain-dev.org)

Suggested Pre-trained Checkpoints

Model	Use	Source
SwinUNETR (MONAI)	Starting point for 3D image encoder	MONAI Model Zoo
nnU-Net v2	Segmentation backbone	github/MIC-DKFZ/nnUNet
MedSAM	Interactive segmentation	github/bowang-lab/MedSAM
DnCNN	Denoising fine-tune base	PyTorch Hub / paper impl
BiomedCLIP	Vision-language for report generation	HuggingFace
GPT-4o / Claude 3.5	Radiology agent LLM	OpenAI API / Anthropic API

Risk Register & Mitigations

Risk	Likelihood	Impact	Mitigation
K-Space microbleed model insufficient sample size	High	Critical	Augment CMB-3D with synthetic phantoms; ensemble with image model; tune for low SNR
Complex Mamba training instability (NaNs)	Medium	High	Use gradient clipping; mixed real/imag training; fall back to real-valued Mamba
LLM hallucination in radiology report	Medium	Critical	Ground all LLM statements in structured finding JSON; add constrained output schema
DICOM diversity: scanner vendor differences	High	High	Test on GE, Siemens, Philips DICOMs; use dcmjs normalisation layer; vendor-specific fixes
Electron app performance: 3D + MRI view	Medium	Medium	Lazy-load MRI volumes; use SharedArrayBuffer for GPU data; profile with Electron
Dataset access delays (CMB-3D requires approvals)	Medium	Medium	Apply for CMB-3D access in Phase 0; use synthetic augmentation as fallback in Phase 1
Single GPU resource for training (Jemin)	Low	Medium	Use Google Colab Pro+ / Vast.ai for large training runs; keep local GPU for fast prototyping

Future Roadmap (Post-MVP)

- **FHIR Integration:** Export reports as HL7 FHIR R4 DiagnosticReport resources for EHR interoperability.
- **Multi-modal Fusion:** Add CT, PET, and DSA (angiography) as additional input modalities to the Foundation Model.
- **Federated Learning:** Train across hospital sites without sharing PHI. Each site updates a shared model.
- **Longitudinal Tracking:** Automatic lesion matching across serial MRI studies. True volume-change timelines.
- **Mobile Companion App:** React Native app for radiologist review and approval on-the-go.
- **Cloud Deployment:** AWS / Azure HIPAA-compliant cloud deployment with Kubernetes for multi-site scaling.
- **Real-time Acquisition AI:** Integration with MRI scanner IPC for real-time K-Space analysis during acquisition.
- **Regulatory Pathway:** FDA 510(k) / CE Mark documentation. Clinical trial planning for diagnostic aid validation.